Conditionals and Iteration

CMP1138: Foundations of Programming

Dr Francesco Del Duchetto
Senior Lecturer in Robotics and Autonomous Systems

University of Lincoln

19 October 2026

Today's Agenda

1. Part 1: Decisions with Conditionals
2. Part 2: Selecting with Switch
3. Part 3: Repeating with Loops
4. Part 4: Live Coding and Recap

Agenda

1. Part 1: Decisions with Conditionals
2. Part 2: Selecting with Switch
3. Part 3: Repeating with Loops
4. Part 4: Live Coding and Recap

Learning Aims

By the end of this lecture, you should be able to:

- Use `if`, `if...else`, and `else if` to control program flow.
- Combine Boolean expressions to make decisions.
- Choose between an `if` chain, the ternary operator, and a `switch`.
- Use `while`, `do...while`, `for`, and `foreach` loops appropriately.
- Trace loop execution and avoid infinite loops.
- Combine conditionals and loops in a small console application.

Review of Week 3 Quiz

Complete the Week 3 recap poll

`https://forms.cloud.microsoft/e/fA0t3yT74L`



From Expressions to Decisions

Last week, expressions produced values, including Boolean values.

```
bool isAdult = age >= 18;
bool receivesDiscount = isStudent || age < 18;
```

A conditional uses a Boolean result to choose which statements run.

Program flow

When a condition is `true`, one path runs. When it is `false`, the program skips that path or follows an alternative one.

Why We Need Conditionals

Programs rarely follow exactly one path from beginning to end.

Scenario	Decision
Online checkout	If payment succeeds, send the order; otherwise, show an error.
Cinema kiosk	If the visitor is a student or under 18, apply a discount.
Smartwatch	If heart rate is high while the user is resting, show an alert.
Cash machine	If the balance covers the withdrawal, dispense cash; otherwise, reject it.

The Basic `if` Statement

```
int age = 20;

if (age >= 18)
{
    Console.WriteLine("You are eligible to vote.");
}
```

- The condition inside parentheses must evaluate to `true` or `false`.
- The block runs only when the condition is `true`.
- Use braces even for one statement: they make later changes safer and clearer.

Reading an `if` Statement

1. Evaluate the condition.
2. If the result is `true`, run the statements inside the braces.
3. If the result is `false`, skip those statements and continue afterwards.

Example

If `score` is 68, then `score >= 40` is true, so a pass message can be displayed.

Adding an Alternative: `if...else`

Use `if...else` when exactly one of two alternatives should run.

```
int age = 16;

if (age >= 18)
{
    Console.WriteLine("Adult ticket");
}
else
{
    Console.WriteLine("Child ticket");
}
```

The `else` block runs only when the `if` condition is false.

More Than Two Outcomes: `else if`

Test the most specific or highest threshold first.

```
int score = 68;

if (score >= 70)
{
    Console.WriteLine("Distinction");
}
else if (score >= 40)
{
    Console.WriteLine("Pass");
}
else
{
    Console.WriteLine("Fail");
}
```

Once one condition is true, its block runs and the rest of the chain is skipped.

Nested Decisions and Scope

A nested `if` is an `if` statement inside another block.

```
bool hasTicket = true;
int age = 17;

if (hasTicket)
{
    if (age >= 18)
    {
        Console.WriteLine("Entry permitted");
    }
}
```

Keep nesting shallow

When conditions are closely related, a combined Boolean expression is often easier to read.

Combining Conditions

The logical operators from Week 3 are especially useful in conditionals.

```
int month = 4;

if (month >= 1 && month <= 12)
{
    Console.WriteLine("Valid month number");
}

if (month < 1 || month > 12)
{
    Console.WriteLine("Invalid month number");
}
```

`&&` means both conditions must be true; `||` means at least one condition must be true.

The Ternary Operator ?:

The ternary (or conditional) operator is a compact choice between two values. It is called **ternary** because it has three parts.

```
int value = -7;
int absoluteValue = value >= 0 ? value : -value;

Console.WriteLine(absoluteValue); // 7
```

```
condition ? valueWhenTrue : valueWhenFalse
```

First, evaluate the condition. The expression returns `valueWhenTrue` when it is true; otherwise, it returns `valueWhenFalse`.
Use it for a short, simple value choice. Use `if...else` when each branch needs several statements.

Agenda

1. Part 1: Decisions with Conditionals
2. Part 2: Selecting with Switch
3. Part 3: Repeating with Loops
4. Part 4: Live Coding and Recap

When to Use `switch`

A `switch` selects a path by matching one value against a set of known cases.

- It is useful when one value has several discrete meanings, such as a menu choice, a day number, or a character command.
- It can be easier to scan than a long `else if` chain.
- Use `if` for ranges or unrelated Boolean conditions.

The `switch` Statement

```
int choice = 2;

switch (choice)
{
    case 1:
        Console.WriteLine("Buy ticket");
        break;
    case 2:
        Console.WriteLine("Buy snacks");
        break;
    default:
        Console.WriteLine("Invalid choice");
        break;
}
```

- `case` labels are possible matches.
- `break` exits the `switch` after a matching case runs.
- `default` handles every value not matched by another case.

Multiple Cases, One Action

Several cases can share the same statements.

```
string day = "Sat";

switch (day)
{
    case "Sat":
    case "Sun":
        Console.WriteLine("Weekend");
        break;
    default:
        Console.WriteLine("Weekday");
        break;
}
```

This is a good fit when several inputs have the same outcome.

Modern `switch` Expressions

A switch expression chooses a value rather than a block of statements.

```
int dayNumber = 6;

string dayName = dayNumber switch
{
    1 => "Monday",
    2 => "Tuesday",
    3 => "Wednesday",
    4 => "Thursday",
    5 => "Friday",
    6 => "Saturday",
    7 => "Sunday",
    _ => "Invalid day"
};
```

The underscore _ is the fallback pattern, similar to `default`.

Agenda

1. Part 1: Decisions with Conditionals
2. Part 2: Selecting with Switch
3. Part 3: Repeating with Loops
4. Part 4: Live Coding and Recap

Why We Need Loops

Repeating similar statements manually is tedious and error-prone.

Without a loop

```
Console.WriteLine(1);
Console.WriteLine(2);
Console.WriteLine(3);
Console.WriteLine(4);
Console.WriteLine(5);
```

With a loop

```
int number = 1;
while (number <= 5)
{
    Console.WriteLine(number);
    number++;
}
```

A loop repeats a block while its Boolean condition permits it.

The Parts of a Reliable Loop

Every loop needs a clear answer to four questions.

Part	**Question**
Initial value	What value does the loop start with?
Condition	When should the loop continue?
Body	What work repeats each time?
Update	What changes so the condition will eventually become false?

The `while` Loop

A `while` loop checks its condition before every iteration.

```
int count = 1;

while (count <= 5)
{
    Console.WriteLine($"Count: {count}");
    count++;
}
```

- Use `while` when the number of repetitions is not known in advance.
- If the condition is false initially, the body does not run at all.
- Make sure the body changes something that can end the loop.

Countdown Exercise

Write a `while` loop that counts down and then prints a final message.

```
int countdown = 5;

while (countdown > 0)
{
    Console.WriteLine(countdown);
    countdown--;
}

Console.WriteLine("Blast off!");
```

Trace it

Which line stops the loop from running forever?

The `do...while` Loop

A `do...while` loop runs the body first and checks the condition afterwards.

```
int number;

do
{
    Console.Write("Enter a number from 1 to 10: ");
    number = int.Parse(Console.ReadLine()!);
}
while (number < 1 || number > 10);
```

Even if the first input is valid, the prompt must run once. That makes `do...while` useful for input validation.

`while` or `do...while`?

Feature	`while`	`do...while`
Condition check	Before the body	After the body
Runs at least once?	No	Yes
Typical use	Retry only while needed	Prompt at least once, then validate

The `for` Loop

A `for` loop keeps the counter setup, continuation condition, and update together.

```
for (int number = 1; number <= 5; number++)
{
    Console.WriteLine(number);
}
```

- `int number = 1` runs once before the loop starts.
- `number <= 5` is checked before each iteration.
- `number++` runs after each iteration.
- Use `for` when you know the counter and range in advance.

Example: Sum from 1 to N

```
Console.Write("Enter N: ");
int maximum = int.Parse(Console.ReadLine()!);
int sum = 0;

for (int number = 1; number <= maximum; number++)
{
    sum += number;
}

Console.WriteLine($"Sum = {sum}");
```

If the user enters 5, the result is 15 because the loop adds 1 + 2 + 3 + 4 + 5.

The `foreach` Loop

`foreach` visits each item in a sequence without managing a counter, condition, or update yourself.

```
string message = "Hi!";

foreach (char character in message)
{
    Console.WriteLine(character);
}
```

- A string is a sequence of `char` values.
- The loop body runs once for each item, in order.
- Use `foreach` when you need each item but not its position.
- Arrays and lists will make this more useful next week.

Infinite Loops and `break`

An infinite loop never makes its condition false.

```
int count = 1;

while (count <= 5)
{
    Console.WriteLine(count);
    // Missing count++: the condition never changes.
}
```

`break` exits the current loop immediately, but it should not replace a clear loop condition without a good reason.

`break` and `continue` with `while`

```
int number = 0;

while (number < 5)
{
    number++;
    if (number == 3)
    {
        continue; // Skip the rest of this iteration.
    }
    if (number == 5)
    {
        break; // Exit the loop entirely.
    }
    Console.WriteLine(number);
}
```

The output is `1`, `2`, and `4`. Update before `continue`, or `number` would stay at 3 forever.

Nested Loops

A nested loop is a loop inside another loop. It is useful for rows, columns, and grids.

```
for (int row = 1; row <= 3; row++)
{
    for (int column = 1; column <= 5; column++)
    {
        Console.Write($"{row * column}\t");
    }

    Console.WriteLine();
}
```

The inner loop completes all five columns for each new row.

Agenda

1. Part 1: Decisions with Conditionals
2. Part 2: Selecting with Switch
3. Part 3: Repeating with Loops
4. Part 4: Live Coding and Recap

Live Coding: Vending Machine v2

Start with last week's change calculator and turn it into a repeatable vending-machine program.

- Keep serving customers until the user chooses to quit.
- Let the customer select from a small set of products.
- Reject an invalid product choice and ask again.
- Check whether the payment covers the selected product.
- Reuse last week's UK-coin change calculation when payment is sufficient.

Vending Machine v2: Control-Flow Plan

Feature	Use it for
`while (true)`	Serve another customer after each completed transaction.
`do...while`	Keep prompting until the product choice is valid.
`switch`	Convert a product name into its price.
`if...else`	Decide whether payment is enough and whether change is due.
`break` / `continue`	Quit the machine or start the next transaction early.

Vending Machine v2: Build It Together

Add the remaining behaviour inside the `while` loop.

1. Display the selected product and its price, then ask for the amount paid.
2. If payment is too small, display the shortfall and use `continue` to start a new order.
3. Otherwise, calculate the change in whole pence.
4. Reuse the week 3 coin algorithm, from £2 down to 1p.
5. Print a coin line only when its count is greater than zero.
6. Challenge: add a `cancel` product option that returns to the choice prompt without ending the machine.

Week 4 Recap Poll

Complete the Week 4 recap poll

`https://forms.cloud.microsoft/e/VXMhT6QeL8`



Summary and What's Next

Today we learned

Next Lecture

Arrays and lists: storing and processing collections of values.

Questions?